ASSIGNMENT 1 FRONT SHEET

Qualification	**BTEC Level 5 HND Diploma in Computing**		
Unit number and title	Prog102: Procedural Programming		
Submission date		**Date Received 1st submission**	
Re-submission Date		**Date Received 2nd submission**	
Student Name	Đặng Hùng Cường	**Student ID**	GCS210714
Class	GCS1007	**Assessor name**	Huỳnh Thị Mỹ Trang
Student declaration I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I understand that making a false declaration is a form of malpractice.			
		Student's signature	

Grading grid

P1	P2	P3	M1	M2	D1

❒ Summative Feedback:		❒ Resubmission Feedback:
Grade:	Assessor Signature:	Date:
Lecturer Signature:		

1 Introduction

Introduce about procedural programming :

Procedural programming is a programming paradigm that uses a linear or top-down approach. It relies on procedures or subroutines to perform computations.

Procedural programming is also known as imperative programming.

In procedural programming, a program consists of data and modules/procedures that operate on the data. The two are treated as separate entities. In the object-oriented programming (OOP) paradigm, however, a program is built from objects. An object is an instance of a class, which is an encapsulation of data (called fields) and the procedures (called methods) that manipulate them. In most, but not all, cases, the fields can only be accessed or modified through the methods. An object therefore is like a miniature program or a self-contained component, which makes the OOP approach more modularized and thus easier to maintain and extend.

Another type of programming paradigm that procedural programming can be contrasted with is event-driven programming. In this approach, procedures are called/executed only in response to events, which may include mouse clicks, keyboard press, attaching or removing a device, arrival of data from an external source, etc. As these events are unpredictable, the procedures that handle them cannot be executed linearly as is the case with procedural programming.

• **General concepts :**

Procedural Programming may be the first programming paradigm that a new developer will learn. Fundamentally, the procedural code is the one that directly instructs a device on how to finish a task in logical steps. This paradigm uses a linear top-down approach and treats data and procedures as two different entities. Based on the concept of a procedure call, Procedural Programming divides the program into procedures, which are also known as routines or functions, simply containing a series of steps to be carried out.

Simply put, Procedural Programming involves writing down a list of instructions to tell the computer what it should do step-by-step to finish the task at hand.

• **Characteristics :**

Based on the concept of a procedure call, Procedural Programming divides the program into procedures, which are also known as routines or functions, simply containing a series of steps to be carried out.

Problem statement

This is the scoring program, if you have more than or equal 3.0, you passed but if you have less than 3.0, you failed.
Step 1: Read user input as data .
Step 2: Check if data is greater than 3.0 or not.
Step 3: if data is greater, console out Passed. Else, console out Failed.

2 Analysis

Analyse the problem and explain:
- List data types, data structures needed in the problem Eight Data Structures

1. Arrays
   - An array is a collection of similar data elements stored at contiguous memory locations. It is the simplest data structure where each data element can be accessed directly by only using its index number.
2. Linked Lists
3. Stacks
4. Queues
5. Hash Tables
6. Trees
7. Heaps
8. Graphs

- Conditional statements needed in the problem:

1 If statement :

The if statement is the first condition a programmer uses to open the ground of the conditional statements. if syntax is as simple as writing if with open and close braces, followed by the condition the programmer intends to compare or check. The if expression simply compares whether the condition (or conditions) enclosed in the braces are true or false.

if true, the ifcode block executes. If false, the execution moves to the next block to check.

```
if (day == " Monday") // Condition
    {
      // Decision
      Console.WriteLine("Go to School");
    }
```

2 Else if statement:

While the if statement can be used to check one condition, else if is used to check multiple conditions. The else if statement (or elif in Python), has syntax similar to the if statement, then followed by the else if block.

For example:

```
if (myDay == " Sunny")
    {
      // Decision
      Console.WriteLine("Read in the Library");
    }
    // 2nd condition
    else if (myday == "Raining")
    {
      //decision
      Console.WriteLine("Read at Home")
    }
    // 3rd condition
    else if (MyDay == "Cloudy")
    {
      // Decision
      Console.WriteLine("Read in the Garden")
    }
```

In else if statements, the conditions are checked from the top-down, if the first block returns true, the second and the third blocks will not be checked, but if the first if block returns false, the second block will be checked. This checking continues until a block returns a true outcome.

3 Else statement

The else statement is the default statement of all the conditional expressions, in all programming languages. That is, when all the if and else if conditions return false (from top to bottom), then the final (default) else block statement executes. The else statement syntax is simply writing else followed by the default statement in open and close curly braces.

```
using namespace Conditional;

static void Main(string[] args)
{
  // This method determines what a user should do
  // for the day based on the weather condition

  public void Weather(string myDay)
  {
    // 1st condition
    if (myDay == " Sunny")
    {
      // Decision
      Console.WriteLine("Read in the Library");
    }
    // 2nd condition
    else if (myday == "Raining")
    {
      //decision
      Console.WriteLine("Read at Home")
    }
    // 3rd condition
    else if (MyDay == "Cloudy")
    {
      // Decision
      Console.WriteLine("Read in the Garden")
    }
```

4 Switch-case

Switch is another version of the conditional statement. It makes codes cleaner and more readable than the conventional if, else if, and else conditional statements.

In switch expressions, each block is terminated by a break keyword. The statements in switch are expressed with cases. For clarity, let’s use a switch statement to illustrate our previous example:

```
int myDay = 4; // setting the value to test
switch (myDay)
{
  case 1:
    Console.WriteLine("Read in the Library");
    break;
  case 2:
    Console.WriteLine("Read at Home");
    break;
  case 3:
    Console.WriteLine("Read in the Garden");
    break;
  default:
    Console.WriteLine("Get some Sleep");
    break;
}
```

switch statements code example.

As previously stated, the switch conditional expression is a cleaner way of writing conditional statements.

Each case is tested until a true value is returned, then the code execution hits the break keyword.

If all the cases return false, the default block is executed.

Therefore, from the code example above, the default value will be executed since none of the test cases 1, 2, and 3 return true.

(The true case in this expression is myDay = 4;.)

Source: https://www.educative.io/answers/what-are-conditional-statements-in-programming

- Loop statement needed in the problem

1 For Loop:

In a for loop, the initial value is performed only once, then the condition tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.

A for loop is a more efficient loop structure in 'C' programming. The general structure of for loop syntax in C is as follows:

**Syntax of For Loop in C:*

```
for (initial value; condition; incrementation or decrementation )
{
  statements;
}
```

-The initial value of the for loop is performed only once.

-The condition is a Boolean expression that tests and compares the counter to a fixed value after each iteration, stopping the for loop when false is returned.

-The incrementation/decrementation increases (or decreases) the counter by a set value.

2 While loop:

In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed.

A while loop is the most straightforward looping structure. While loop syntax in C programming language is as follows:

**Syntax of While Loop in C:*

```
while (condition) {
             statements;
}
```

It is an entry-controlled loop. In while loop, a condition is evaluated before processing a body of the loop. If a condition is true then and only then the body of a loop is executed. After the body of a loop is executed then control again goes back at the beginning, and the condition is checked if it is true, the same process

is executed until the condition becomes false. Once the condition becomes false, the control goes out of the loop.

After exiting the loop, the control goes to the statements which are immediately after the loop. The body of a loop can contain more than one statement. If it contains only one statement, then the curly braces are not compulsory. It is a good practice though to use the curly braces even we have a single statement in the body.

In while loop, if the condition is not true, then the body of a loop will not be executed, not even once. It is different in do while loop which we will see shortly.

3 Do-While Loop

In a do…while loop, the condition is always executed after the body of a loop. It is also called an exit-controlled loop.

A do…while loop in C is similar to the while loop except that the condition is always executed after the body of a loop. It is also called an exit-controlled loop.

Syntax of do while loop in C programming language is as follows:

**Syntax of Do-While Loop in C:*

```
 do {
  statements
} while (expression);
```

As we saw in a while loop, the body is executed if and only if the condition is true. In some cases, we have to execute a body of the loop at least once even if the condition is false. This type of operation can be achieved by using a do-while loop.

3 Design

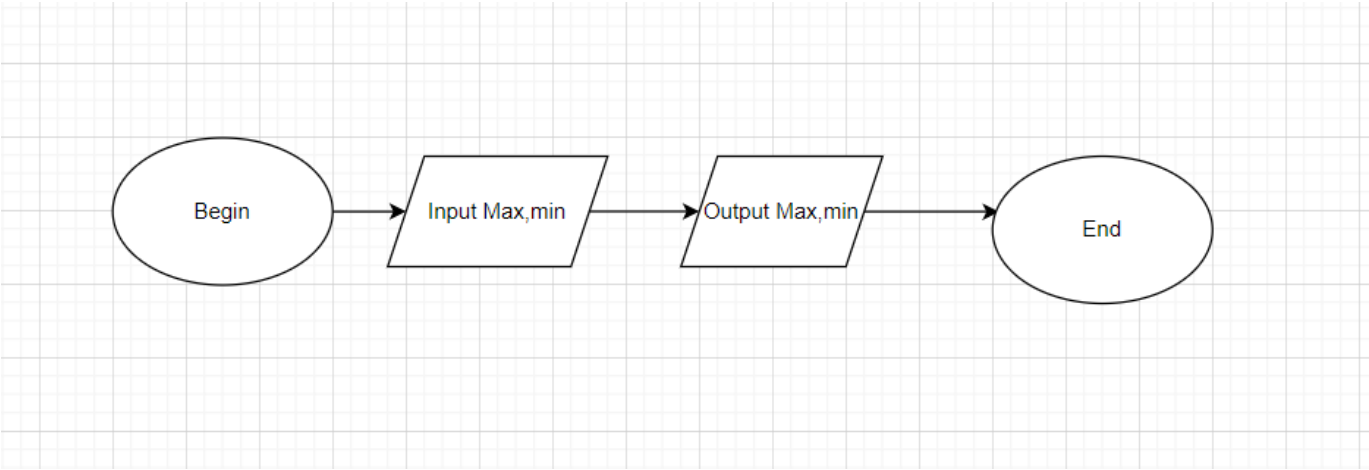
3.1 WBS

A work breakdown structure (WBS) is a visual, hierarchical and deliverable-oriented deconstruction of a project. It is a helpful diagram for project managers because it allows them to break down their project scope and visualize all the tasks required to complete their projects.

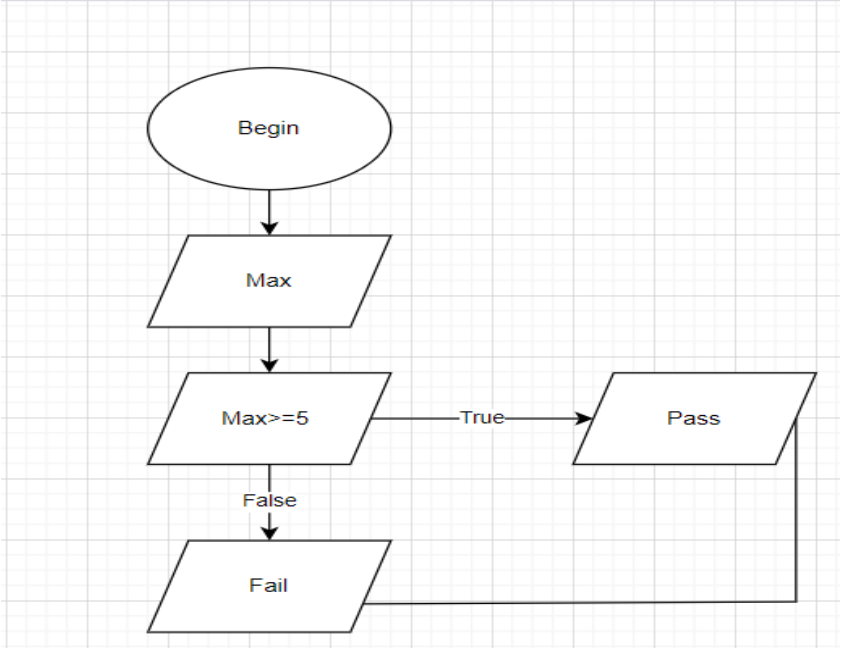
3.2 Flowcharts

-Flowcharts of each small algorithm and explanation.

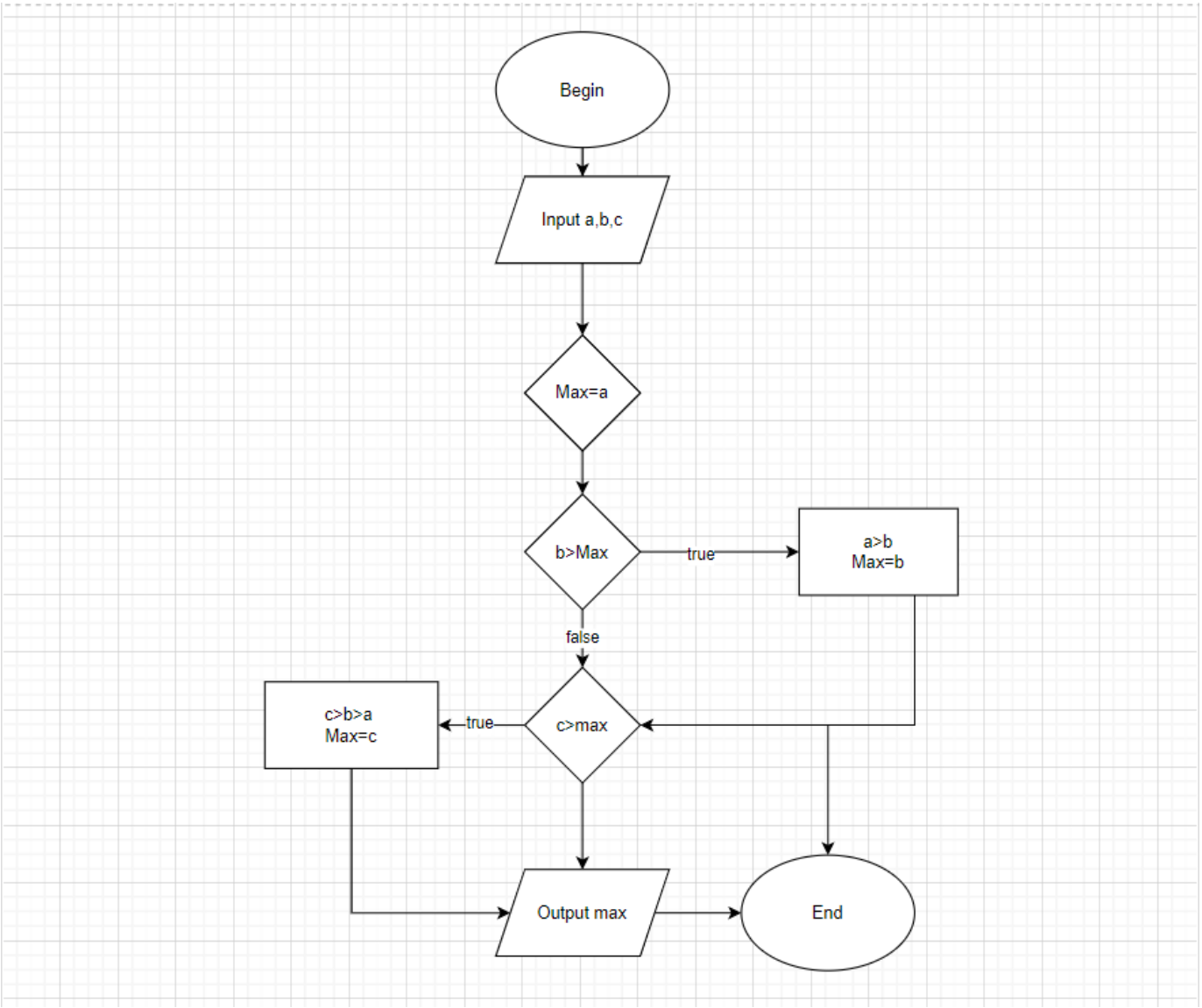
-This is a flowchart about menu operation



-This is Flowchart about pass or fail



-This is Flowchart about max



-This is Flowchart about min

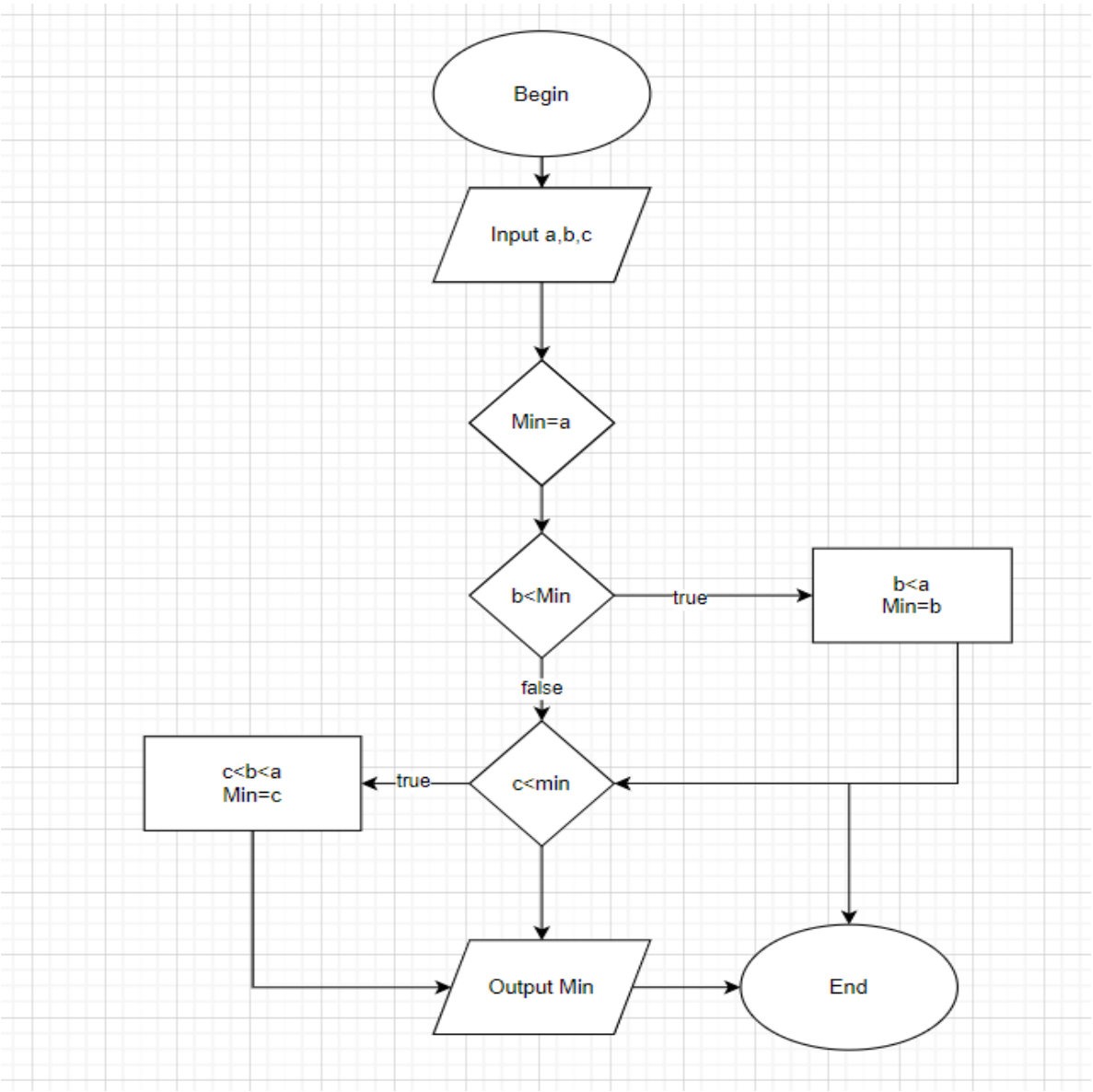


Table of content

Introduction……2

Analysis……3

List data types, data structures needed in the problem Eight Data Structures……3

If statement……..3

Else if statement……..3

Else statement……..4

Switch-case……...6

For Loop……..8

While loop……8

Do while loop……..9

3 Design……10

3.1 WBS……10

3.2 Flowcharts……10,11,12.

-End-